

Logging

Version: 1.0

Purpose

This document defines the logging architecture of Trading Platform Pro and its primary application, the Trading Cockpit.

Logging is a core infrastructure capability for:

- operational transparency
- diagnostics
- troubleshooting
- runtime supervision
- broker investigation
- market data investigation
- order lifecycle traceability
- reconciliation
- controlled incident analysis

Logging shall observe application behaviour without changing business behaviour.

Objectives

The logging system shall provide:

- structured logging
- deterministic event naming
- consistent event context
- explicit operational state
- traceable trading workflows
- PAPER and LIVE context
- broker lifecycle visibility
- market data lifecycle visibility
- order lifecycle visibility
- reconciliation visibility
- controlled exception logging
- sensitive data protection
- configurable retention
- acceptable runtime overhead

Logging Principles

Logging shall be:

- structured
- deterministic
- explicit
- concise
- contextual
- searchable
- operationally meaningful
- secure

Logs shall explain:

- what happened
- which capability was affected
- which business or technical identity was involved
- what the resulting state is
- whether operational impact exists

Logging shall not become a substitute for persisted business state.

Logging Responsibility

Infrastructure provides the logging capability.

Domain and Application components may emit meaningful events or logging context through approved abstractions.

Logging infrastructure shall not:

- change Domain state
- make trading decisions
- submit orders
- retry failed operations
- repair reconciliation discrepancies

Logging observes behaviour.

Structured Logging

Operationally relevant logs shall use structured fields.

Prefer:

```
```text
event=ORDER_SUBMISSION_REQUESTED
order_id=order-123
instrument_id=instrument-456
environment=PAPER
```
```

Avoid:

```
```text
Submitting some order for instrument...
```
```

Free-form messages may supplement structured context but shall not replace critical fields.

Event Names

Operational event names shall use deterministic uppercase snake case.

Examples:

```
```text
RUNTIME_STARTING
RUNTIME_OPERATIONAL
BROKER_CONNECTING
BROKER_CONNECTED
MARKET_DATA_DISCONNECTED
ORDER_SUBMISSION_REQUESTED
ORDER_TRANSMITTED
ORDER_ACKNOWLEDGED
ORDER_REJECTED
ORDER_EXECUTED
RECONCILIATION_DISCREPANCY_DETECTED
```
```

Avoid inconsistent variants such as:

```
```text
Order submitted
ORDER_SUBMIT
submitOrder
order_sent
```
```

One operational meaning shall use one canonical event name.

Event Naming Rules

Event names shall:

- describe a completed observation or explicit operation state
- use capability language
- remain stable where practical
- avoid provider-specific terminology unless logging provider-specific adapter behaviour

Examples:

Application-oriented:

```
```text
```

```
ORDER_ACKNOWLEDGED
```

```
```
```

Provider-specific diagnostic event:

```
```text
```

```
IB_ORDER_STATUS_RECEIVED
```

```
```
```

Provider-specific events shall not replace application-oriented lifecycle events.

Log Levels

Supported log levels:

| Level | Purpose |
|-------|---------|
|-------|---------|

| | |
|-----|-----|
| --- | --- |
|-----|-----|

| | |
|-------|---------------------------------|
| DEBUG | Detailed diagnostic information |
|-------|---------------------------------|

| | |
|------|---------------------------|
| INFO | Normal operational events |
|------|---------------------------|

| | |
|---------|--|
| WARNING | Unexpected or degraded but recoverable state |
|---------|--|

| | |
|-------|--------------------------------|
| ERROR | Operation or capability failed |
|-------|--------------------------------|

| | |
|----------|--|
| CRITICAL | Safe continued operation may be affected |
|----------|--|

Log level shall reflect operational impact.

A business rejection is not automatically a technical error.

Example:

```
```text
```

```
ORDER_REJECTED
```

```
```
```

may be `WARNING` when the broker explicitly rejects an order and the application remains operational.

Standard Context Fields

Structured logging may use standard context fields including:

- `event`
- `component`
- `operation`
- `runtime\_state`
- `environment`
- `profile`
- `correlation\_id`
- `duration\_ms`
- `timestamp`

Only relevant fields shall be emitted.

Avoid empty structured fields solely for schema completeness.

Trading Context Fields

Trading workflow logs may include:

- `instrument\_id`
- `symbol`
- `candidate\_id`
- `decision\_id`
- `order\_id`
- `position\_id`
- `portfolio\_id`
- `review\_id`

Business identifiers shall use canonical internal identities where available.
Provider identifiers may be included as additional diagnostic fields.

Broker Context Fields

Broker logs may include:

- `broker\_provider`
- `broker\_connection\_state`
- `broker\_order\_id`
- `broker\_execution\_id`
- `broker\_timestamp`
- `account\_reference`

Broker-specific identifiers shall remain clearly named.

Do not log broker credentials.

Account references shall be masked where required.

Market Data Context Fields

Market data logs may include:

- `market\_data\_provider`
- `market\_data\_state`
- `subscription\_id`
- `instrument\_id`
- `source\_timestamp`
- `freshness\_state`

Market data logs shall preserve the distinction between:

- current
- stale
- unavailable
- disconnected

PAPER and LIVE Context

Trading-related operational logs shall expose execution environment where relevant.

Use:

```
```text
environment=PAPER
```
```

or:

```
```text
environment=LIVE
```
```

LIVE order lifecycle events shall always contain explicit LIVE context.

The execution environment shall not be inferred from broker endpoint details during log analysis.

Runtime Logging

Runtime lifecycle events shall be logged.

Canonical runtime events may include:

```text

RUNTIME\_BOOTSTRAPPING

RUNTIME\_INITIALIZING

RUNTIME\_STARTING

RUNTIME\_OPERATIONAL

RUNTIME\_DEGRADED

RUNTIME\_STOPPING

RUNTIME\_STOPPED

RUNTIME\_FAILED

```

Runtime state transitions shall include:

- previous state where relevant
- new state
- affected capability where relevant
- failure context where relevant

Startup Logging

Startup logs shall include non-sensitive operational context.

Examples:

```text

APPLICATION\_STARTING

CONFIGURATION\_LOADED

CONFIGURATION\_VALIDATED

DEPENDENCY\_CONTAINER\_BUILT

INFRASTRUCTURE\_INITIALIZED

APPLICATION\_SERVICES\_INITIALIZED

PRESENTATION\_INITIALIZED

WORKSPACE\_RESTORED

RUNTIME\_OPERATIONAL

```

Startup logs may include:

- application version
- active profile
- PAPER or LIVE environment
- broker provider
- market data provider

Secrets shall never be logged.

Configuration Logging

Configuration logging shall record:

- active profile
- configuration validation result
- configuration source types where useful
- non-sensitive operational configuration context

Do not log complete configuration objects.

Do not log:

- credentials
- API keys
- access tokens
- passwords
- secret environment variables

Configuration validation failures shall identify the affected configuration field without exposing secret values.

Broker Lifecycle Logging

Broker lifecycle shall remain observable.

Canonical broker events may include:

```text

BROKER\_INITIALIZED  
BROKER\_CONNECTING  
BROKER\_CONNECTED  
BROKER\_DEGRADED  
BROKER\_DISCONNECTED  
BROKER\_RECONNECTING  
BROKER\_CONNECTION\_FAILED  
BROKER\_STOPPING  
BROKER\_STOPPED  
```

Connection logs shall include:

- broker provider
- environment
- connection state
- attempt number where relevant

Reconnection logs shall not imply repeated order submission.

Market Data Lifecycle Logging

Market data lifecycle shall remain observable.

Canonical events may include:

```text

MARKET\_DATA\_INITIALIZED  
MARKET\_DATA\_CONNECTING  
MARKET\_DATA\_CONNECTED  
MARKET\_DATA\_DEGRADED  
MARKET\_DATA\_DISCONNECTED  
MARKET\_DATA\_CONNECTION\_FAILED  
MARKET\_DATA\_STOPPING  
MARKET\_DATA\_STOPPED  
```

Market data connection state shall remain distinguishable from broker connection state.

Subscription Logging

Market data subscription lifecycle may use:

```text

MARKET\_DATA\_SUBSCRIPTION\_REQUESTED  
MARKET\_DATA\_SUBSCRIPTION\_ACTIVE  
MARKET\_DATA\_SUBSCRIPTION\_STALE  
MARKET\_DATA\_SUBSCRIPTION\_FAILED  
MARKET\_DATA\_SUBSCRIPTION\_CLOSING  
MARKET\_DATA\_SUBSCRIPTION\_CLOSED  
```

Subscription logs shall include where relevant:

- subscription identity
- instrument identity
- provider

- source timestamp
- freshness state

High-frequency quote updates shall not automatically produce INFO logs.

High-Frequency Data Logging

High-frequency market data paths require controlled logging.

Do not log every quote update at INFO level.

Detailed quote logging may be enabled at DEBUG level for controlled diagnostics.

Logging shall avoid:

- excessive allocations
- uncontrolled file growth
- UI degradation
- event loop degradation

Operational state changes remain more important than individual data messages.

Trading Candidate Logging

Trading Candidate lifecycle may use:

```text

```
TRADING_CANDIDATE_CREATED
TRADING_CANDIDATE_REVIEW_STARTED
TRADING_CANDIDATE_ACCEPTED
TRADING_CANDIDATE_REJECTED
TRADING_CANDIDATE_ARCHIVED
```
```

Candidate logs should include:

- candidate identity
- instrument identity
- origin where relevant
- resulting candidate state

Candidate notes or user-entered text shall not automatically be copied into operational logs.

Trading Decision Logging

Trading Decision lifecycle may use:

```text

```
TRADING_DECISION_CREATED
TRADING_DECISION_ACCEPTED
TRADING_DECISION_REJECTED
TRADING_DECISION_CANCELLED
TRADING_DECISION_EXECUTED
TRADING_DECISION_REVIEWED
```
```

Decision logs should include:

- decision identity
- candidate identity where available
- instrument identity
- resulting decision state

An accepted decision shall not be logged as an executed order.

Decision and order lifecycle events remain separate.

Order Lifecycle Logging

Order lifecycle logging is business-critical.

Canonical order events may include:

```
```text
ORDER_CREATED
ORDER_VALIDATED
ORDER_VALIDATION_FAILED
ORDER_SUBMISSION_REQUESTED
ORDER_SUBMISSION_BLOCKED
ORDER_TRANSMITTED
ORDER_ACKNOWLEDGED
ORDER_PARTIALLY_FILLED
ORDER_FILLED
ORDER_CANCELLATION_REQUESTED
ORDER_CANCEL_PENDING
ORDER_CANCELLED
ORDER_REJECTED
ORDER_EXECUTION_RECEIVED
```
```

Event names shall preserve lifecycle distinctions.

Do not use one generic `ORDER\_SUBMITTED` event for multiple states.

Order Submission Logging

Order submission logs shall distinguish:

1. Application submission request
2. Local validation
3. Transmission attempt
4. Broker acknowledgement
5. Broker rejection
6. Execution state

Example:

```
```text
ORDER_SUBMISSION_REQUESTED
ORDER_VALIDATED
ORDER_TRANSMITTED
ORDER_ACKNOWLEDGED
```
```

A successful adapter method call shall not automatically produce `ORDER\_ACKNOWLEDGED`. Broker acknowledgement requires broker-derived acknowledgement state.

Order Submission Context

Order submission logs may include:

- `order\_id`
- `instrument\_id`
- `environment`
- `submission\_key`
- `action`
- `quantity`
- `order\_type`

Price parameters may be logged when operationally required and not sensitive.

Logs shall not become the authoritative order record.

Persisted order state remains authoritative.

Duplicate Submission Logging

Duplicate-prevention behaviour shall remain observable.

Canonical events may include:

```
```text
ORDER_DUPLICATE_CHECK_STARTED
ORDER_DUPLICATE_CHECK_PASSED
ORDER_DUPLICATE_SUBMISSION_BLOCKED
```
```

Logs should include:

- order identity
- submission key where applicable
- environment
- blocking reason

The logging system shall not perform duplicate detection.

The Application workflow owns duplicate-prevention rules.

Order Retry Logging

Order submission shall not be blindly retried.

If an explicitly approved order workflow performs a retry, logs shall include:

- retry attempt
- previous failure state
- retry reason
- order identity
- submission identity

Canonical event example:

```
```text
ORDER_SUBMISSION_RETRY_REQUESTED
```
```

Logging a retry does not authorize the retry.

Retry policy belongs to the Application workflow.

Order Rejection Logging

Broker rejection shall remain explicit.

Example:

```
```text
event=ORDER_REJECTED
order_id=order-123
broker_provider=interactive_brokers
environment=PAPER
```
```

Rejection context may include:

- normalized rejection reason
- provider error code
- provider error reference

Sensitive provider payloads shall not be logged blindly.

Execution Logging

Execution lifecycle may use:

```
```text
ORDER_EXECUTION_RECEIVED
ORDER_PARTIALLY_FILLED
ORDER_FILLED
```
```

Execution logs may include:

- order identity

- broker order identity
- broker execution identity
- filled quantity
- remaining quantity
- execution price
- broker timestamp
- environment

Execution logs shall preserve broker-derived timestamps where available.

Position Lifecycle Logging

Position lifecycle may use:

```text

POSITION\_OPENED  
POSITION\_UPDATED  
POSITION\_CLOSING  
POSITION\_CLOSED  
```

Position logs should include:

- position identity
- instrument identity
- resulting position state
- environment where relevant

Position lifecycle events shall not be inferred solely from UI state.

Reconciliation Logging

Reconciliation is operationally critical.

Canonical reconciliation events may include:

```text

RECONCILIATION\_STARTED  
RECONCILIATION\_STATE\_LOADED  
RECONCILIATION\_DISCREPANCY\_DETECTED  
RECONCILIATION\_NO\_DISCREPANCY  
RECONCILIATION\_ACTION\_REQUIRED  
RECONCILIATION\_COMPLETED  
RECONCILIATION\_FAILED  
```

Reconciliation logs may include:

- local state reference
- broker state reference
- discrepancy classification
- affected capability
- affected identity

Reconciliation logs shall not silently change business state.

Reconciliation Discrepancies

Discrepancies shall remain explicit.

Example context:

```text

event=RECONCILIATION\_DISCREPANCY\_DETECTED  
capability=position  
position\_id=position-123  
classification=STATE\_MISMATCH  
environment=PAPER

---

Avoid embedding complete broker payloads in free-form log messages.  
Persist detailed evidence separately when required by the reconciliation workflow.

---

## Workspace Logging

Workspace lifecycle may use:

```text

WORKSPACE_LOADING
WORKSPACE_RESTORED
WORKSPACE_RESTORE_FAILED
WORKSPACE_FALLBACK_LOADED
WORKSPACE_PERSISTED
WORKSPACE_PERSIST_FAILED

```

Workspace logging shall not contain business data snapshots.  
Widget layout state is presentation state.

---

## UI Logging

UI logging should focus on operationally relevant failures and state transitions.

Do not log:

- every button click
- every mouse movement
- every table selection
- trivial widget repaint activity

Relevant UI events may include:

```text

PRESENTATION_INITIALIZED
WIDGET_INITIALIZATION_FAILED
CRITICAL_NOTIFICATION_PRESENTED

```

User trading actions shall be represented through Application workflow events rather than duplicate UI-only logs.

---

## Exception Logging

Exceptions shall be logged at the boundary that owns failure handling.

An exception should normally be logged once.

Exception logs shall include:

- canonical event name
- component
- operation
- relevant identity
- operational impact
- exception context

Stack traces shall be preserved for unexpected technical failures.

Avoid logging the same exception repeatedly across every architecture layer.

---

## Expected Failures

Expected failures shall use meaningful operational events.

Examples:

```text

ORDER_REJECTED

```
BROKER_DISCONNECTED
MARKET_DATA_UNAVAILABLE
---
```

Expected operational failures do not always require stack traces.
Use stack traces for unexpected technical failures.

Unexpected Failures

Unexpected failures shall preserve technical context.

Example:

```
```text
RUNTIME_COMPONENT_FAILED

```

Relevant fields may include:

- component
- operation
- runtime state
- correlation id
- exception type

Unexpected failures shall not terminate the application silently.

---

## Correlation

Correlation identifiers may be used for multi-step workflows.

Examples:

- order submission workflow
- reconciliation cycle
- runtime startup
- controlled shutdown

A correlation identifier shall remain stable for one logical workflow.

Do not use one global correlation identifier for the complete application session when finer workflow identity is required.

---

## Correlation Example

Example order workflow:

```
```text
correlation_id=workflow-abc
event=ORDER_SUBMISSION_REQUESTED
correlation_id=workflow-abc
event=ORDER_TRANSMITTED
correlation_id=workflow-abc
event=ORDER_ACKNOWLEDGED
---
```

The canonical `order\_id` remains the business identity.

The correlation identifier supports workflow tracing.

Timestamps

Structured logs shall include application log timestamps.

Time-dependent external events may additionally include:

- `broker\_timestamp`
- `source\_timestamp`

Timestamp semantics shall remain explicit.

Logs shall not silently mix:

- local application time
 - broker time
 - market data source time
- Timezone context shall be defined consistently.
-

Sensitive Data

Never log:

- passwords
- API keys
- authentication tokens
- broker credentials
- secret environment variables
- private keys

Sensitive values shall be omitted or masked.
Do not rely solely on developer discipline.
Logging infrastructure should support redaction where practical.

Personal and Account Information

Personal information shall not be logged unless explicitly required for a defined operational or regulatory purpose.
Account references shall be masked where practical.
Avoid logging complete external account payloads.
Example masked value:
```text  
account\_reference=\*\*\*\*1234  
```

Provider Payloads

Raw provider payloads shall not be logged automatically.
Provider payloads may contain:

- credentials
- account information
- undocumented fields
- personal information
- excessive data

Provider diagnostics shall use normalized structured fields.
Controlled DEBUG diagnostics require explicit review.

Log Configuration

Logging configuration is managed centrally.
Configuration may include:

- log level
- output destination
- structured format
- file rotation
- retention policy
- DEBUG diagnostics

Logging configuration shall not change trading behaviour.
LIVE logging defaults shall avoid excessive high-frequency diagnostic output.

Log Output

The primary log format should support structured processing.
JSON Lines may be used for structured file logging.

Example:

```
```json
{
 "event": "ORDER_ACKNOWLEDGED",
 "order_id": "order-123",
 "environment": "PAPER"
}
```
```

Human-readable console output may use a compact formatted representation.
Canonical event names and structured fields shall remain preserved.

Performance

Logging shall remain efficient.

Logging shall:

- avoid unnecessary string formatting
- avoid expensive serialization in disabled log levels
- avoid uncontrolled high-frequency output
- avoid blocking the UI thread
- avoid blocking the AsyncIO event loop where practical

Business-critical events shall not be dropped silently solely for performance optimization.

Log Retention

Log retention shall be configurable.

Retention policy may define:

- retention period
- maximum file size
- rotation strategy
- archival behaviour

Retention shall consider:

- operational investigation
- storage limits
- security
- sensitive information exposure

Retention requirements may differ between development, PAPER and LIVE environments.

Audit-Relevant Events

Business-critical events may require audit-oriented persistence or logging.

Examples:

- LIVE environment activation
- order lifecycle state changes
- reconciliation discrepancies
- configuration changes affecting trading capability
- security-relevant changes

Audit requirements shall be defined explicitly.

Operational logs are not automatically sufficient as a regulatory audit record.

Audit Logging

Where audit logging is implemented, audit records should be:

- append-oriented
- traceable
- timestamped
- identity-aware
- protected from silent modification where practical

Audit records shall preserve relevant business identity.

Audit logging shall remain separate from mutable UI state.

Logging Tests

Critical logging behaviour requires automated tests.

Tests shall verify where relevant:

- canonical event names
- required context fields
- PAPER and LIVE context
- order lifecycle distinctions
- broker acknowledgement distinction
- duplicate submission blocking events
- reconciliation discrepancy events
- exception logging ownership
- secret redaction
- account masking
- high-frequency log suppression
- timestamp semantics

Tests shall not depend on LIVE trading.

Logging Evolution

Logging evolves with concrete operational workflows.

Before introducing a new canonical event:

1. Identify the operational meaning.
2. Check for an existing equivalent event.
3. Identify the owning capability.
4. Define required context.
5. Define log level.
6. Evaluate sensitive data.
7. Add tests where business-critical.
8. Update documentation.

Avoid multiple event names for the same operational state.

Log Review Checklist

Before introducing or changing logging verify:

- operational purpose identified
- canonical event name defined
- capability owner identified
- log level appropriate
- required business identity included
- environment included where relevant
- broker state distinguished from local state
- market data state distinguished from broker state
- order lifecycle state precise
- broker acknowledgement not inferred
- duplicate submission risk observable
- reconciliation state observable

- correlation used where useful
- timestamp semantics explicit
- sensitive data excluded
- account information masked where required
- provider payload not logged blindly
- high-frequency impact evaluated
- automated tests added
- documentation synchronized

Related Documents

- Product_Vision.md
- Product_Roadmap.md
- Project_Overview.md
- Architecture.md
- Infrastructure.md
- Technical_Specifications.md
- Configuration.md
- Runtime.md
- Monitoring.md
- API_Guidelines.md
- Testing_Strategy.md
- Security.md
- AGENTS.md